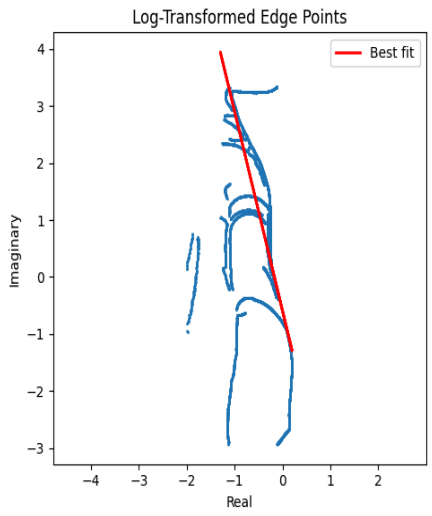
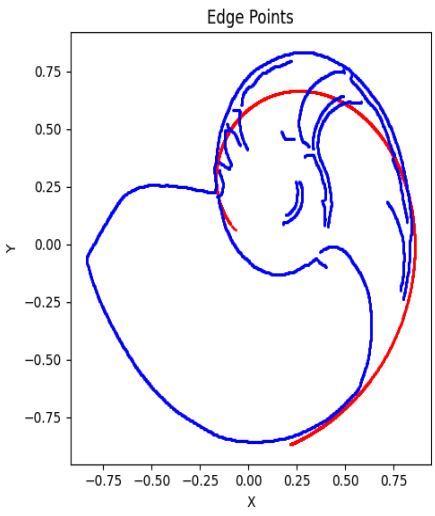
How can Principal Component Analysis be used to detect the Golden Ratio in images?

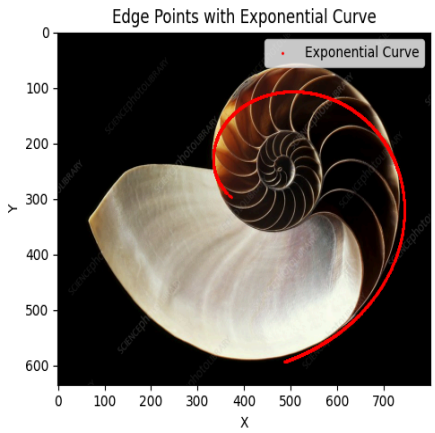
Introduction

The Golden Ratio has always been presented as something extraordinary. Dating back to 300 BCE, discovered by Euclid, its self-similarity has been renowned for centuries. But often I see artworks where the Golden ratio is loosely fit on top of the image as an example of the Golden Spiral appearing in art or nature. I remember my Latin class, where the front page of our work packet had the same spiral on it. Seeing this mathematical constant talked about so poetically made me question: Is it possible to quantifiably determine whether the Golden ratio is actually present in an image?

This investigation aims to determine whether spiral-like structures in images can be detected and analysed mathematically by transforming image data into a form where logarithmic spirals become linear structures.







Outline

My first thought was to perform some kind of regression, using the image as data points, where the function type is a golden spiral. The regression would optimize parameters that control transforms such as rotation, scaling, and inversion. Common things you would find across images that may contain the Golden Ratio. But this regression would get messy quickly, seeing as the golden spiral is non-linear. So, some kind of linearization, potentially through logarithms, would help turn our regression into a much simpler linear regression model. And we'd also need a way of getting data points, or in other words, points of interest, from our image. But our data may be too noisy or non-convex for just vanilla regression, so PCA (Principal Component Analysis) can be used instead to find the dominant trend in our 2d data. And also to help cluster our data and cull out noisy areas that interfere with PCA. I went searching online and found some alleged examples of the Golden Spiral in nature, and I intend to benchmark how well they actually match the Golden Ratio

Defining the Golden Spiral.

The Golden Ratio is defined exactly as :

$$\phi = \frac{\left(1 + \sqrt{5}\right)}{2}$$

It is the positive solution to:

$$x = 1 + \frac{1}{x}$$

In fact, this is the only positive number that satisfies the equation; not only this, but its definition gives it the property such that its two smaller parts (a) and (b) have the same ratio as their combined parts compared to the largest of its smaller parts. This self-similar property allows the creation of patterns such as the Fibonacci sequence:

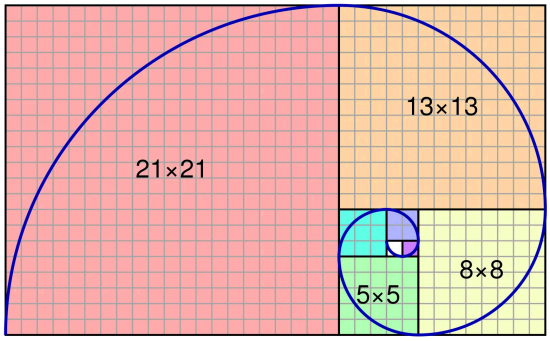
$$a_{n+1} = a_n + a_{(n-1)}$$

And as n approached infinity, this ratio:

$$\frac{a_{n+1}}{a_n}$$

Approaches the Golden Ratio.

The adjacent Fibonacci spiral, also known as the Golden Spiral, is simply a curve constructed from squares with areas proportional to the terms in the Fibonacci sequence.



But, instead of using the terms within the Fibonacci sequence to approximate the Golden Spiral, we can simply use the Golden Ratio itself to generate the subsequent square. But we need a more precise definition since we want to detect this curve within images. So, how do you define this new spiral exactly?

Well, we see it's not a regular function since one value of x can yield multiple values of y; it is not injective. But that's only true in the case of a Cartesian function. If instead we use a polar function where the radius (r) is defined by the angle (θ). We know the ratio of consecutive Fibonacci numbers approaches a single value, which means the sequence as a whole approaches a geometric sequence. Meaning the values grow exponentially. This is going to be helpful in our definition since now we know the radius grows exponentially with our angle. So, $r \propto n^{\theta}$ we could use any value of n, but since e has special properties that make it easy to work with later, we'll define n as Euler's number (e). But this only describes a general logarithmic spiral; we need the radius of the spiral to increase by ϕ every quarter turn, which in radians is π/2. So if we add another parameter called b that controls the growth rate, we get:

$$r = e^{b\theta} \quad \text{or expanded} \quad r = \left(e^{b}\right)^{\theta}$$

If we define (b) as $b = \ln(\phi)$, it increases by the Golden Ratio every full turn, as $r = \phi^{\theta}$. But to get the true Golden Spiral, we need to define b as:

$$b = \frac{\ln(\phi)}{\frac{\pi}{2}}$$

Since it must increase by ϕ every quarter turn. Then, there we have our full definition:

$$r = e^{b\theta} \quad \text{Where:} \quad b = \frac{\ln(\phi)}{\frac{\pi}{2}}$$

But polar form is hard to work with, especially when working with 2d images that have pixels that live in the Cartesian plane. To convert this back into the Cartesian plane, we can use the exponential form of our function:

$$z = re^{i\theta}$$

We already have a definition of (r), and our angle is a parameter, so we can leave it as be. After substituting in (r), we get:

$$z(\theta) = e^{b\theta}e^{i\theta}$$ Then simplifying $$z(\theta) = e^{(b+i)\theta}$$

This now implies that our image will live in the complex plane.

Linearization

We've defined our logarithmic function as:

$$z(\theta) = e^{(b+i)\theta}$$

But now, we need parameters for transforms such as rotation and scaling. So we'll update its definition as follows:

$$z(\theta) = ae^{(b+i)\theta} \cdot e^{ic}$$

Then expanding

$$z(\theta) = ae^{b\theta+i\theta+ic}$$

And factoring:

$$z(\theta) = ae^{b\theta} \cdot e^{i(\theta+c)}$$

Where (a) represents scaling and (c) represents rotation about the axis. We won't worry about translation because we can simply center the spiral within our image at the origin later.

For our model, we need to somehow linearize this data. My first thought was, since it's an exponential function, we can use a logarithm to linearize it. But the output is a complex

number, so we need to extend the domain of our logarithm to: z ∈ ℂ. If we represent z in exponential form, we can use basic logarithm rules to define the complex logarithm:

$$\ln\left(re^{i\theta}\right)$$

Expanding:

$$\ln(r) + \ln\left(e^{i\theta}\right)$$

Simplifying:

$$\ln(r) + \theta i$$

So generally:

$$\ln(z) = \ln\left(|z|\right) + i\arg(z)$$

But the arg function is multivalued since every time you rotate 2pi you return to the same point, so this function is actually multivalued:

$$\ln(z) = \ln\left(|z|\right) + i\left(\arg(z) + 2\pi k\right)$$

Where k ∈ Z

So we need to define which range of the function we'll look at (which branch), it doesn't matter too much since each branch contains the same information, just shifted vertically. So we'll use the standard principal branch where k=0 and the range goes from [-π, π].

If we look at the definition, we see the x-axis represents the log of the radius of our points, and the y-axis represents the angle.

Applying this to our definition, $z(\theta)$ we get:

$$\ln\left(z(\theta)\right) = \ln(a) + b\theta + (\theta + c)i$$

Now we can separate our real and imaginary components and treat them as our x and y values. Thus:

$$x = \ln(a) + b\theta \quad y = \theta + c \quad \theta = y - c$$

Substituting θ:

$$x = \ln(a) + b\theta$$

$$x = \left(\ln(a) - bc\right) + by$$

Writing in terms of y:

$$y = \frac{1}{b}x - \left(\frac{\ln(a)}{b} - c\right)$$

This tells us that a rotation parameter is not needed since any change in rotation (c) can be compensated for with scale (a). Since (c) only affects vertical shifts in log space and never the slope, the same applies to (a).

$$z(\theta) = ae^{b\theta} \cdot e^{i(\theta + c)}$$

$$z_2(\theta) = \left(e^{-cb}\right)e^{b\theta} \cdot e^{i(\theta + 0)}$$

These represent the same curve for any value of (c), showing that (a) can compensate for rotation. To reconstruct our spiral, from the linearized data, we only need the slope and y-intercept. This also tells us that if the spiral is truly a golden spiral, it will have a slope of 1/b. Where b is:

$$b = \frac{\ln(\phi)}{\frac{\pi}{2}}$$

.

Image Processing

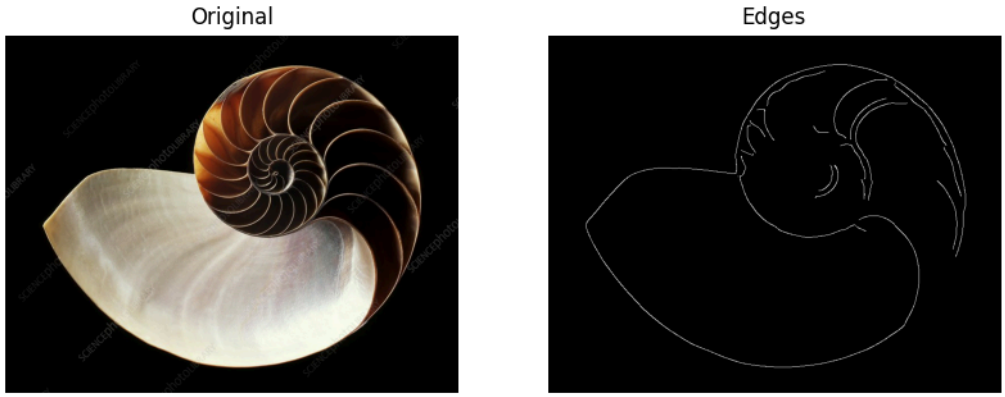
Our images will be defined as a matrix of RGB values with dimensions:

Width x Length x 3

With that 3 being for our red, green, and blue values, which store color information. But this matrix is massive, even a low-resolution image of size 256 x 256 x 3 will have 196,608 datapoints.

We need to reduce this number and focus only on the points that retain the most meaning. For this, I decided on edge detection since it removes all the redundant data with low contrast, meaning pixels that are portraying the same information.

Since we're only focused on contrast, we can collapse our color dimension by averaging our red, green, and blue values into a single value called luminance. Helping to reduce datapoints in combination with edge detection:



Detected edges in white.

Specifically, we're using the Canny Edge Detector. It works by approximating the gradient of the image in the x and y directions, using a 2D Convolution, which is a series of dot products with a predefined kernel. These gradients are used to approximate the magnitude and orientation of the edges. Along with other processing steps that are beyond the scope of this investigation.

$$M = \sqrt{G_x^2 + G_y^2} \quad \text{and} \quad D = \arctan 2\left(G_x, G_Y\right)$$

The Canny Edge Detector also includes a hyperparameter (σ), which controls how strict it is when including smaller lines. By increasing this value, we can help to decrease noise. This is especially gonna be helpful if I want to test more artistic images like the ones I often see. We'll also perform another 2D convolution, known as a Gaussian Blur, before doing edge detection to help smooth out smaller edges that are just noise that could interfere with our model. Now we'll take only the pixel coordinates where there is an edge present, and convert each into a complex number using their pixel coordinates: $z_i = x_i + iy_i$. Now our image is represented by a list of complex numbers.

We'll also divide each complex number by the image size so that the maximum magnitude of a component cannot be greater than 1. This is known as normalization, and it helps standardize images with varying resolutions. Images will range from [-1, 1] in their real and imaginary components

Centering the Spiral

If our curve is off-center by some value C, then all of our linearization math no longer works:

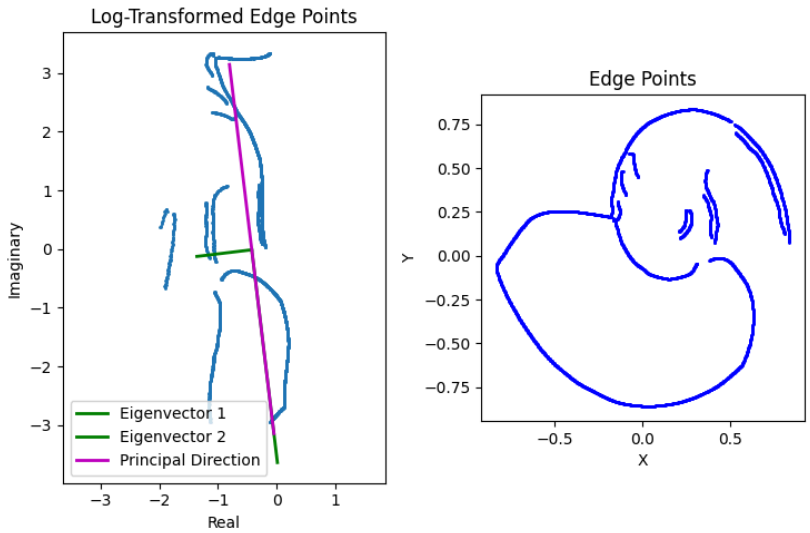
$$z(\theta) = ae^{b\theta} \cdot e^{i(\theta)} + C$$

That means we must find the center of the image or a point near it to approximate the linearization better. This means we'll need a metric that describes how linear the logarithm centered at a given point is. This is where Principal Component Analysis (PCA) comes in. PCA allows us to determine the principal directions of our datapoints, essentially the directions with the greatest and least spread. If the spread of the data when we apply the log centered at C has a high anisotropy, the ratio of the spread in its two principal directions, then the data is relatively linear, meaning that C is a close approximation. Although there are many methods, such as regression, to locate the best C, since we only have two axes, the parameter space isn't that large. We can just check a lot of possible positions and choose the one that appears the most linear.

First, we calculate the covariance matrix, which encodes the general spread of our data:

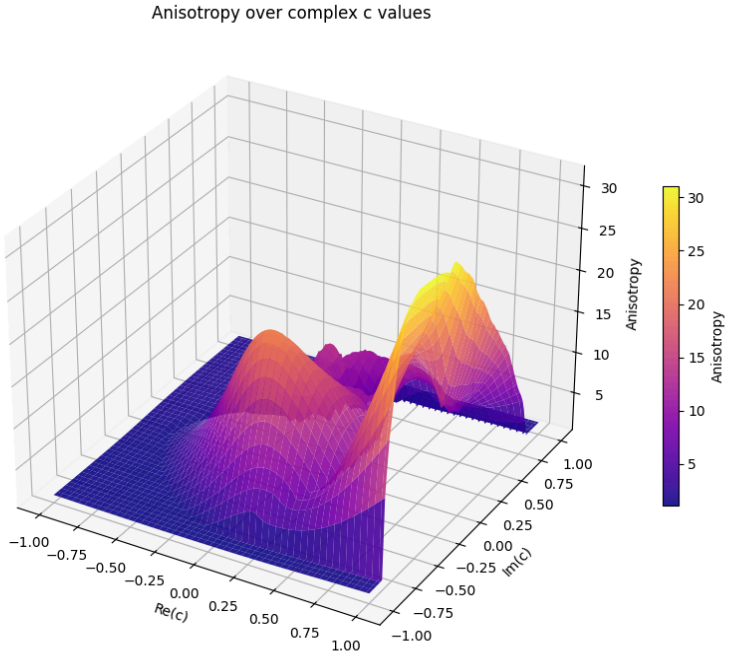
$$\begin{array}{c} \\ x \\ y \end{array} \begin{array}{c} \begin{array}{cc} x & y \end{array} \\ \begin{bmatrix} var(x) & cov(x,y) \\ cov(x,y) & var(y) \end{bmatrix} \end{array}$$

Then, by calculating the eigenvectors of the covariance matrix, we get the directions in which to rotate so that none of our points are rotated and only scaled. Also associated with each eigenvector is an eigenvalue, which describes the scalar by which our points are transformed. This eigenvalue indicates the overall spread of our data in that direction.

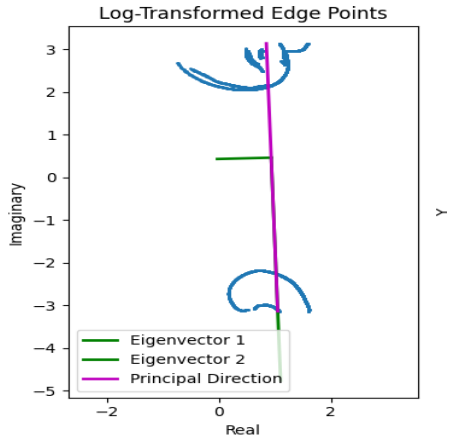


Principal Direction is the direction with the most spread, which is the same as the largest eigenvector

So formally, our metric is anisotropy, which is defined as $a = \frac{\lambda_2}{\lambda_1}$, where λ_2 is the principal direction. I decided to create, using matplotlib in Python, a 3D graph where our z-axis is anisotropy, and our x and y are our C value's coordinates relative to the image:



There's a clear peak in the true center of the spiral, but there's also another peak at the extreme x-values. When plotting at the log-transformed points at this peak, we see it's not truly very linear:



What's happening is our metric only cares that one axis has a high spread while the other has a relatively low spread, and these discontinuous blobs of points meet this criterion. This type of grouped distribution is called bimodal, and there are plenty of metrics that detect it. The one that

gave me the best results in my testing was the Bimodality Coefficient, which is described by this equation:

$$b = \frac{\gamma^2 + 1}{k}$$

Where γ is the skew and k represents kurtosis.

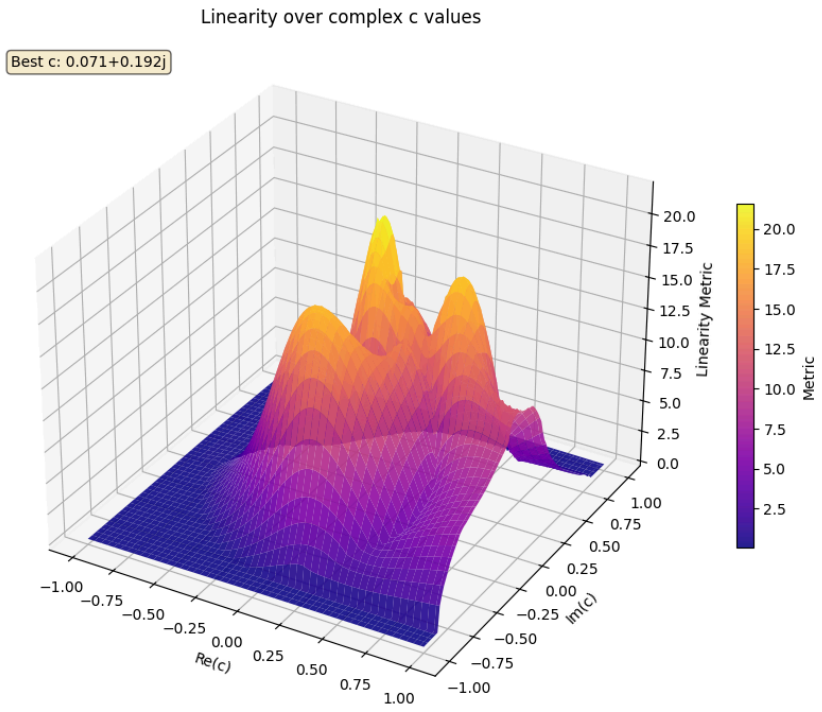
This function ranges from 0 to 1 and is near 1 when the data is bimodal. So, using this, we can construct our new metric for our z-axis:

$$a = \frac{\lambda_2}{\lambda_1} \qquad b = \frac{\gamma^2 + 1}{k}$$

Then:

$$m_{etric} = a \cdot (1 - b)$$

Finally, we're going to update our definition of anisotropy to prioritize longer lines, since the spiral trend is generally the longest in the image. $a = \frac{\lambda_2^2}{\lambda_1}$



The graph now peaks near the true center

Reconstructing our spiral:

Now that we have a center, we can linearize our data using it, then use PCA to get the direction of our linear trend. And create a line passing through the centroid of our log-transformed points. Recall that our linearization step was:

$$\ln(z - C) + C$$

So to convert back from log space into pixel space, we can use this inversion equation:

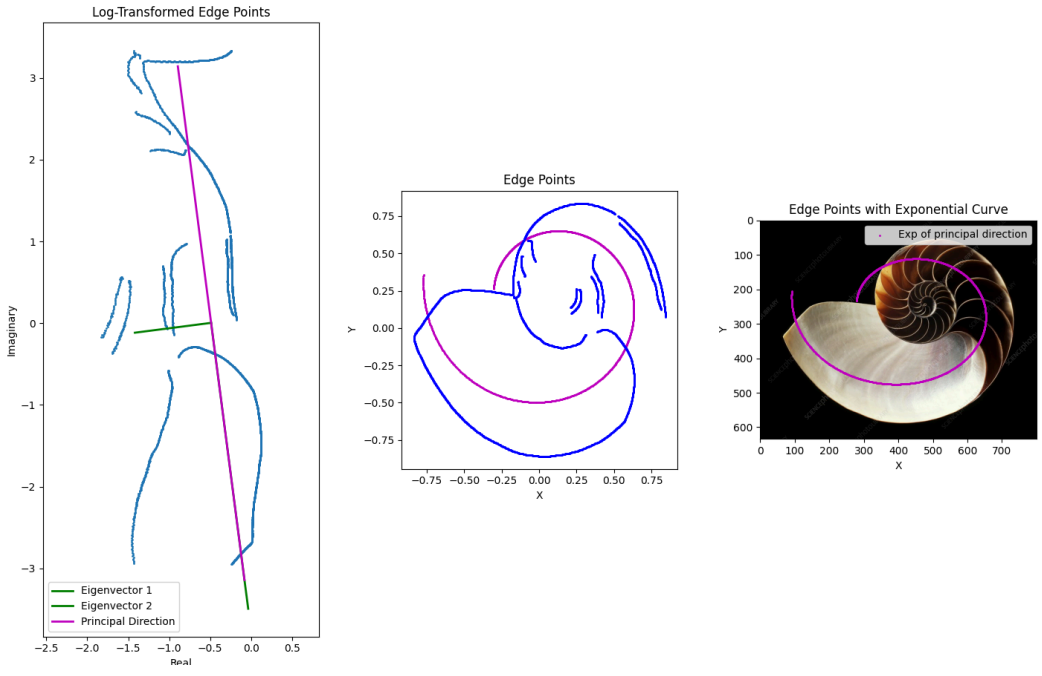
$$e^{(z' - c)} + C$$

Where z-prime is constructed by using the vector of the Principal Direction for the slope, and it must pass through the center, so in point slope form, it must be:

$$(x - C_x)m = y - C_x$$

$$m = \frac{E[2]}{E[1]}$$

Where E is the larger eigenvector, or our Principal Direction. Now that we have our line, we can plug it into our inverse equation, which will convert it back into a spiral, which gives us:



Our calculated slope was $m = -7.7$, and $b = \frac{1}{m}$ so the (b) value was -0.129. But for the golden spiral, the b-value must be ± 0.306. Suggesting this spiral is near the golden ratio, but not truly.

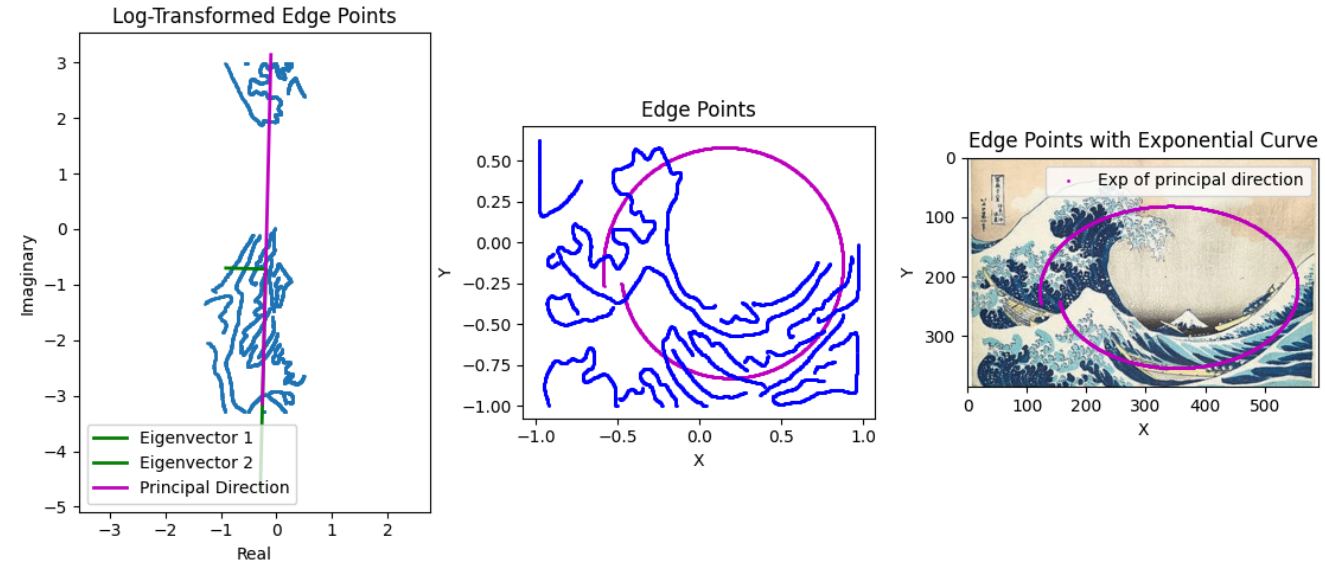
Reflection

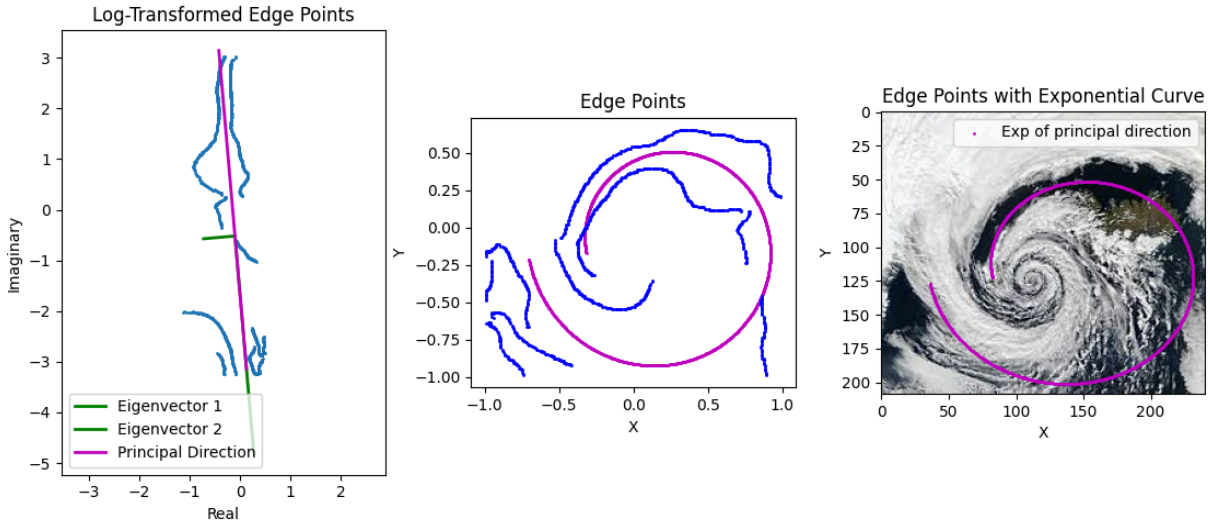
Throughout this investigation, I had to overcome many hurdles and problems with theory vs. real-world data. In a perfect world, the images wouldn't be noisy and would have clear spiral patterns, but real-world data has noise, which forced me to account for that in my math. For example, I struggled with coming up with a good metric for linearity since often the real data would be so noisy that by chance a random center could appear linear according to my metric. So I had to add redundancy to handle harder images in my dataset. And I had to purposely avoid using algorithms that I wouldn't be able to explain briefly, such as noise-redundant regressions, like RANSAC, which could've produced much cleaner results despite the noise.

But by avoiding well-suited algorithms such as Hough Transforms, I ended up with what, in my opinion, is a much more elegant solution. The math feels much more justifiable instead of just a series of operations.

Although, in my opinion, I still think something like a custom Hough Transform that detects spirals would perform much better since it's known for being redundant to noise. I noticed that the more abstract the spiral got, the worse the approximation got.

Here are a few examples:





I also noticed that it was rare that the b-value was ever actually near the proper value for the golden ratio, which I argue is moderate evidence that the golden ratio's prevalence is overstated.

In conclusion, PCA can reliably be used to a moderate extent to detect logarithmic spirals within images. This method included edge detection, linearization, centering, PCA, and reconstruction of the original spiral to robustly detect spiral or circular patterns. My research challenged the idea that the golden ratio appears naturally in art and nature, as my measured values did not match those of a golden spiral. I also noticed in my graphs that the metric surface did not look too noisy for a gradient descent approach, so that's a possible progression for the future. Thinking back to my Latin class, I wonder how the example images that were in our packet would hold up against my metric. Although this method is still limited due to factors such as hyperparameters in places like the Canny Edge Detector, which must be arbitrarily chosen. A natural progression of this research would be implementing better-suited algorithms, such as Hough Transforms, for better reconstruction.